• what is python
→ high-level, general-purpose Programming language
→ created by Guido van Rossum in 1991
→ case sensitive

• machine code → 0,1

• Variable
ex name = "Rahul"
↑ variable ↑ value

= → Assignment opretor

Rules
→ A variable Name can contain Letters.
→ A variable Name can contain Numbers.
ex. 1student → invalid
student1 → valid
→ A variable Name can contain an underscore
→ A variable Name cannot contain spaces
→ Variable Names Are case-sensitive
→ A variable Names cannot Be a python keyword

FEBRUARY 2026

04 WEDNESDAY

WK 06 • DAY 035-330

FEB 2026

| | |
|---|---|
| F | Monday |
| E | Tuesday |
| B | Wednesday |
| 2 | Thursday |
| 0 | Friday |
| 2 | Saturday |
| 6 | Sunday |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | 1 |
| 2 | 9 | 16 | 23 | | | |
| 3 | 10 | 17 | 24 | | | |
| 4 | 11 | 18 | 25 | | | |
| 5 | 12 | 19 | 26 | | | |
| 6 | 13 | 20 | 27 | | | |
| 7 | 14 | 21 | 28 | | | |
| 8 | 15 | 22 | | | | |

09 • Data Types

1. integer → 18, 85, -5
2. float → 5.8, 99.50, -5.00
3. String → "Ranui", "18"
4. Boolean → True, False

11 5. None → None

12 • Arithmetic operators

- + → addition 10+5 = 15
- 01 - → subtraction 10-5 = 5
- * → multiplication 10*5 = 50
- 02 / → Divison 10/5 = 2.0
- // → floor Division 10//3 = 3
- 03 % → Modulo 10%3 = 1
- ** → Exponentiation 2**3 = 8

04 • String

05 we creat string by '...', "...", '''...''', """..."""

06 - string indexing

start from :- 0

p y t h o n
0 1 2 3 4 5 :- positive index
-6 -5 -4 -3 -2 -1 :- negetive index

- How to acces letter
word = "python"
print(word[2]) :- t
print(word[10]) :- index error

- string slicing
word = "python"
print(word[0:3]) :- pyt
print(word[::2] :- pyhn

- operation of string
+ :- string are allow addition with other string

* :- string are allow to multiplication with int

09 • String are immutable

word = "python"

10 word[0] = "J" :- Type error

11 word = "J" + word[1:0] # Jython (redefine)

print(word)

12

• String methods

01

- case conversion.

02 upper() / lower() / capitalize()

title() / swapcase() / casefold()

03

- upper()

04 name = "python"

print(name.upper()) :- PYTHON

05

• lower()

06 name = "TARANG"

print(name.lower()) :- tarang

• capitalize ()

```
text = "python program"
print (text.capitalize())
```

:- Python Program

only ek convert ~~to~~ first latter of string into upper.

• title ()

```
text = "python program lan"
print (text.title())
```

:- Python Program Lan

convert first latter of word into upper

• swapcase ():

```
text = "pytho N"
print (text.swapcase())
```

PYtHOn





SUNDAY 08

FEB 2026
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday

- casefold()

```
text = "HELLO"
print(text.casefold())
```

:- hello

- searching in string
in / not in / find() / index()/count()
startswith() / endswith()

- in / not in

```
massage = "Hello python"
print("python" in massage) :- True
print("python" not in massage) :- False
print("Java" in massage) :- True
print("Java" not in massage) :- False
```

- find()

```
text = "Hello python"
print(text.find("python")) :- 6 (start index)
print(text.find("Java")) :- -1 (not found)
```

- index
It give start index of word

• count()
text = "banana"
print(text.count("a")) :- 3

• startswith()
text = "python Programming"
print(text.startswith("Pa")) :- True

• endswith()
file = "notes.txt"
print(~~text~~.file.endswith(".txt")) :- true

• replacing Text
text = "I like Java"
new = text.replace("Java", "Python")
print(new) :- I like Python

• .strip()
text = " Hello "
print(text.strip()) :- Hello

• .lstrip() / rstrip() ,, left / right

```
text = "   Hello
print(text.lstrip())
print(text.rstrip())
```

• \n (new line) • \t (tab)

```
print("Hello\nworld")   :- Hello
                                world
```

• len()

```
text = " Tarang"
print(len(text))   :- 6
```

• str() :- use to convert into string

```
age = 20
print(type(age))   :- int
age = str(age)
print(type(age))   :- str
```

• formating string

```
name = "Jhon"
age = 20
print(f"My name is {name} i am {age} old")
```

- split()

```
text = "python is easy"
word = text.split()
print(word) :- ['python', 'is', 'easy']
```

- Boolean
  - values :- True, False

:- comparison operators are return value in Boolean

for OR
True True :- True
True False :- True
False True :- True
False False :- False

for and
True True :- True
True False :- False
False True :- False
False False :- False

FEBRUARY 2026

13 FRIDAY

WK 07 • DAY 044-321

FEB 2026

| | | | | | | |
|---|---|---|---|---|---|---|
| Monday | | 2 | 9 | 16 | 23 | |
| Tuesday | | 3 | 10 | 17 | 24 | |
| Wednesday | | 4 | 11 | 18 | 25 | |
| Thursday | | 5 | 12 | 19 | 26 | |
| Friday | | 6 | 13 | 20 | 27 | |
| Saturday | | 7 | 14 | 21 | 28 | |
| Sunday | 1 | 8 | 15 | 22 | | |

not true :- False

not false :- True

- input and output

input() :- takes data from the user
print() :- display data to the user

- input()
  - with message
    name = input("enter your name:- ")
  - multiple input
    a, b = input("enter two num: ").split()
  - int input
    num = int(input("enter a number: "))

default :- string (str):

- float input
  num = float(input("enter price: "))

09 • conditional statements:
• if / • if-else / • if-elif-else

10 • if
11 age = 20
if age >= 18:
print("you are an adult")

12 :- you are an adult

01 • if-else
02 age = 17
if age >= 18:
03 print("you are an adult")
else:
04 print("not adult")

:- not adult

F Monday
E Tuesday
B Wednesday
2 Thursday
0 Friday
2 Saturday
6 Sunday

09 • if - elif - else

```
10 if condition1:
       statement1
11 elif condition2:
       statement2
12 elif condition3:
       statement3
01 else :
       statement4
02
```

example:

```
03 marks = 75
   if marks >= 90:
04     print ("Excellent")
   elif marks >= 60:
05     print ("good")
   elif marks >= 40:
06     print ("pass")
   else:
       print ("fail")
```

if, good

• Nested conditions

```
age = 20
if age >= 18:
    if age <= 60:
        print("between 18 and 60")
    print("above 18")
```

:- between 18 and 60
above 18

• Not

```
is_closed = False
if not is_closed:
    print("the shop is open")
```

:- The shop is open

• and, or

```
if condition1 and condition2

if condition1 or condition2:
```

- for loop

09 • for i in range(1,6):
print(i)

10 :- 1, 2, 3, 4, 5

11 for :- start the loop
i :- loop variable

12 in :- takes values from sequence

range :- provides the values

01 print :- output

02 • for i in range(5):
print("Hello")

03 :- Hello Hello Hello Hello Hello

04 • for i in range(1,10,2) ← step
print(i)

05 1, 3, 5, 7, 9

06 • Negative step
for i in range(5, 0, -1):
print(i)

:- 5, 4, 3, 2, 1

- Nested for loop
outer loop
  - inner loop
  - inner loop
  - inner loop

```
for i in range(3):
    for j in range(2):
        print(i,j)
```

0,0 / 0,1 / 1,0 / 1,1 / 2,0 / 2,1

- Nested for loop for ROWS and columns

```
for row in rang(3):
    for column in range(4):
        print("*", end="")
    print()
```

:-
****
****
****

FEBRUARY 2026

20 FRIDAY

WK 08 • DAY 051-314

FEB 2026: Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday

- slow for loop

```
for row in range(1,5):
    for column in range(1,row+1)
        print(column, end = "")
    print()
```

:-
1
12
123
1234

- while loop

```
while condition:
    statement
```

- i = 1

```
while i <=5:
    print(i)
    i+=1
```

:- 1 / 2 / 3 / 4 / 5

- Three part of while loop
  1. Initialization
  2. Condition
  3. update

```
* * * *
* * * *
* * * *
```

One important key to success is self confidence

```
#initialization
#condition
i=1
while i<= 5:
    print(i)
#update
    i=i+1
```

- if you can't update value it's might be infinite loop

- Enter number
  - ↓
  - Is it 0?
    - yes → ↓ Stop
    - No → | Process → ↓ Ask again

```
num = int(input("enter num:-"))
while num != 0:
    print ("you entered:", number)
    num = int(input("enter :num:"))
print("Loop ended")
```

JANUARY
23 2026
MONDAY
WK 08 • DAY 054-311

FEB 2026
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday

```
num = 472
while number != 0:
    digit = number % 10
    print(digit)
    num = num // 10
```

2
7
4

- example of infinite loop

```
i = 1
while i <= 5:
    print(i)
```

go correcting add i+=1

- nested while loop

```
row = 1
while row <= 3:
    column = 1
    while column <= 4:
        print("*", end="")
    column += 1
    print()
row += 1
```



• for VS while.

for
- for known sequences/ranges
- used with range()
- loop varible changes automatically
- convenient for strings.
- convenient for known iteration counts.

while.
- for condition-controlled repetition.
- uses a Boolen condition.
- programmer usually controls the update
- useful for repested input.
- useful when stopping depends on changing input/data.

FEBRUARY 2026

25 WEDNESDAY

- Match-case.
  - Syntax

```
Match value:
    case pattern 1:
        #code
    case pattern 2:
        #code
    case pattern 3:
        #code
    case _:
        # default code
```

  - choice = 2

```
Match choice:
    case 1:
        Print("Add")
    case 2:
        Print("Subtract")
    case 3:
        Print("Multiply")
    case 4:
        Print("Divide")
    case _:
        Print("Invalid choice")
```

:- subtract

09

- multiple values

```
day = 6
match day:
    case 1|2|3|4|5:
        print("weekday")
    case 6|7:
        print("weekend")
    case _:
        print("Invalid day")
```

- nested match case

outer match
|
|- case "student"
|  |
|  |- Inner match
|  |  |- case 1
|  |  |- case 2
|  |  |- case 3
|
|- case "teacher"
|  |- Inner match
|
|- case _

FEBRUARY 2026

27 FRIDAY

WK 09 • DAY 058-307

FEB 2026

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | 1 |
| 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 9 | 10 | 11 | 12 | 13 | 14 | 15 |

- Monday
- Tuesday
- Wednesday
- Thursday
- Friday
- Saturday
- Sunday

09 • break, continue and pass.
- break → stops the nearest loop.
10 - continue → skip the current iteration.
- pass → Do nothing.
11
• break
12 syntax for variable: in sequence.
if condition:
01 break
• for i in range(1, 6):
02 if i==4
break
03 print(i)
04 ?- 1, 2, 3
05 • while i <= 5:
if i == 4:
06 break
print(i)
i += 1
1, 2, 3

If you want to live a happy life, tie it to a goal, not to people or things

- continue
syntax for variable in sequence:
    if condition:
        continue
    statement

- for i in range(1,6):
    if i==3:
        continue
    print(i)
:- 1/2/4/5

- pass
Do nothing where you don't want any code
write pass

- function

09 - creat function
```
def greet():
    Print("Hello, students!")
```
11 - call function.
```
greet()
```
12

- Parameters and Arguments.

01
```
def greet(name): → Parameter
    Print("Hello", name)
```
02
```
greet("saurabh") → Argument
```
03 • positional Argument
```
def sub(a,b):
    print(a-b)
```
04
```
sub(10,2)
```
05

- return in function.

06
```
def add(a,b):
    return a+b
result = add(10,20)
print(result) :-30
```

- after return code not work in function
- returning multiple values.

```
def cal(a,b):
    total = a+b
    difference = a-b
    return total, difference

answer = cal(10, 4)
print(answer) :- (14, 6)
```

↳ tuple

:- it return value in tuple type

- Keyword Arguments and Default.
  - Default → Default

```
def greet(name = "Guest"):
    print("Hello", name)
greet()          :- Hello, Guest
greet("Aasha") :- Hello Aasha
```

M Monday 30 2
A Tuesday 31 3 9
R Wednesday 4 10 16
Thursday 5 11 17
2 Friday 6 12 18
0 Saturday 7 13 19
2 Saturday 1 8 14 20
6 Sunday 15 21
22

* Multiple default parameters.

```
def introduce(name, city="pune"):
    print(name, "is from", city)
introduce("Asha")
introduce("Rahul", "Mumbai")
```

```
def describe(name, age, city):
    print(name, age, city)
```

```
describe("Asha", age=21, city="pune")
```
:- Asha 21 pune

```
def add(a, b):
    return a+b
```

```
add(5)
```
:- type error : missing required

```
add(5,2,1)
```
:- type error :- to many positional.

```
add(5,2)
```
:- 7

- / :- means parameters before it are positional only

- * :- means parameters after it are keyword only

```
def divide(a,b, /)
    return a/b
print(divide(10,2))   :- 5.0
print(divide(a=10, 2) :- type error

def user(name, *, age, city):
    return f"{name}, age {age}, from {city}"

print(user("Asha", age=20, city="Pune"))
:- Asha age 20 from Pune.

print(user("Asha", 20, "Pune"))
:- type error
```

MARCH 2026

06 FRIDAY

WK 10 • DAY 065 300

*args

:- it take multiple value

```
def ym(*qrgs)
    print(qrgs)
    print(type(qrgs))
ym(1,2,3,4,5,6)
```

:- (1,2,3,4,5,6)

:- type - tupple

**kwargs

```
def show(**kwargs):
    print(kwargs)
    print(type(kwargs))

show(name="Aysha", age=20, city="pyno")
```

:- {'name': 'Aysha', 'age': 20, 'city': 'pyno'}
<class 'dict'>

Overcoming barriers to performance is how groups become teams